Section 2

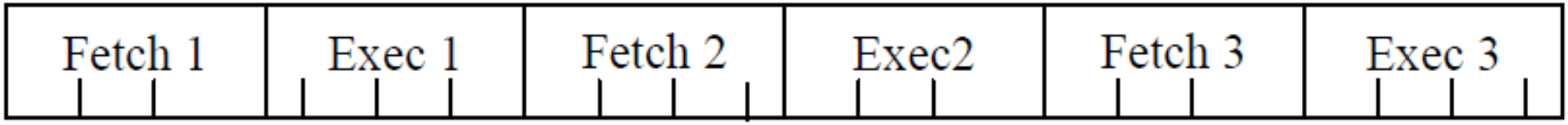
Micro CC421

Pipeline

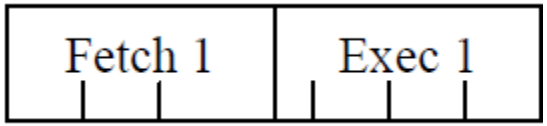
Instruction 1; { Fetch, Exec }

Instruction 2; { Fetch, Exec }

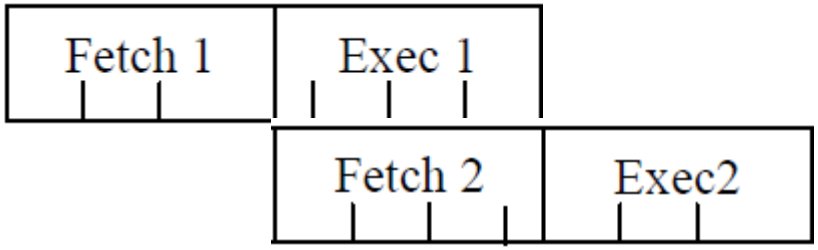
Instruction 3; { Fetch, Exec }



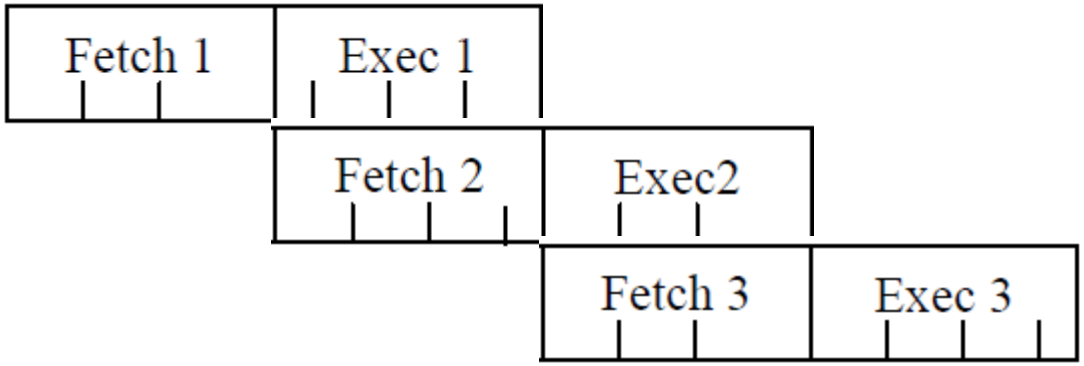
Pipeline

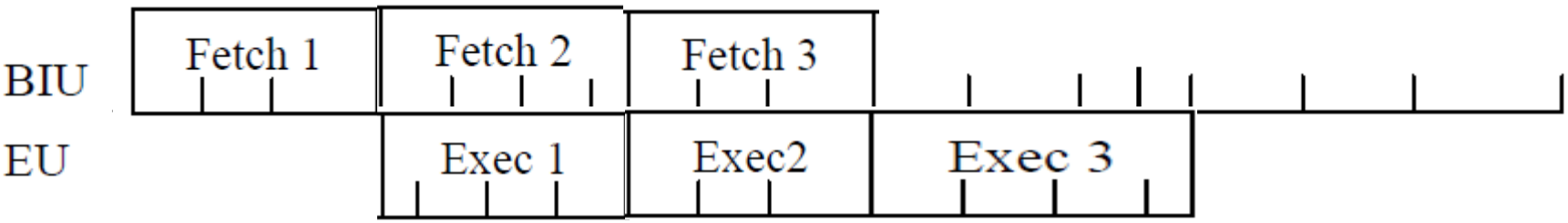


Pipeline



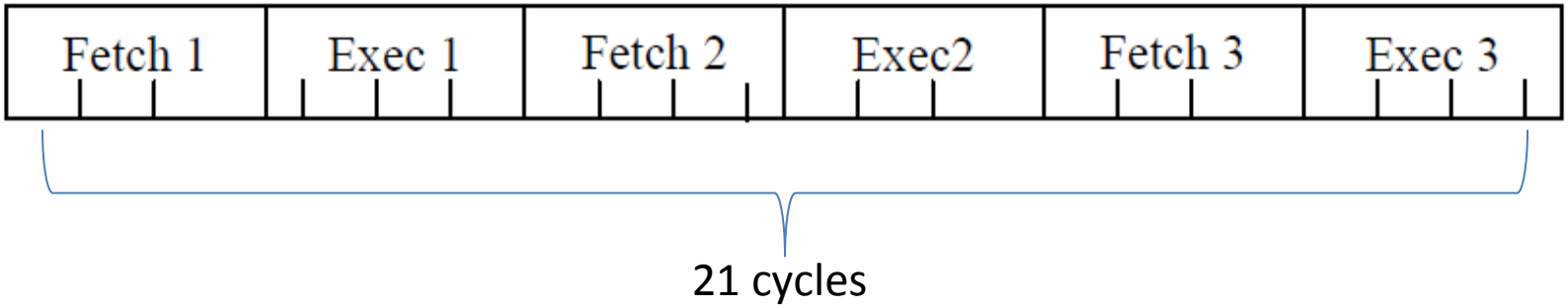
Pipeline

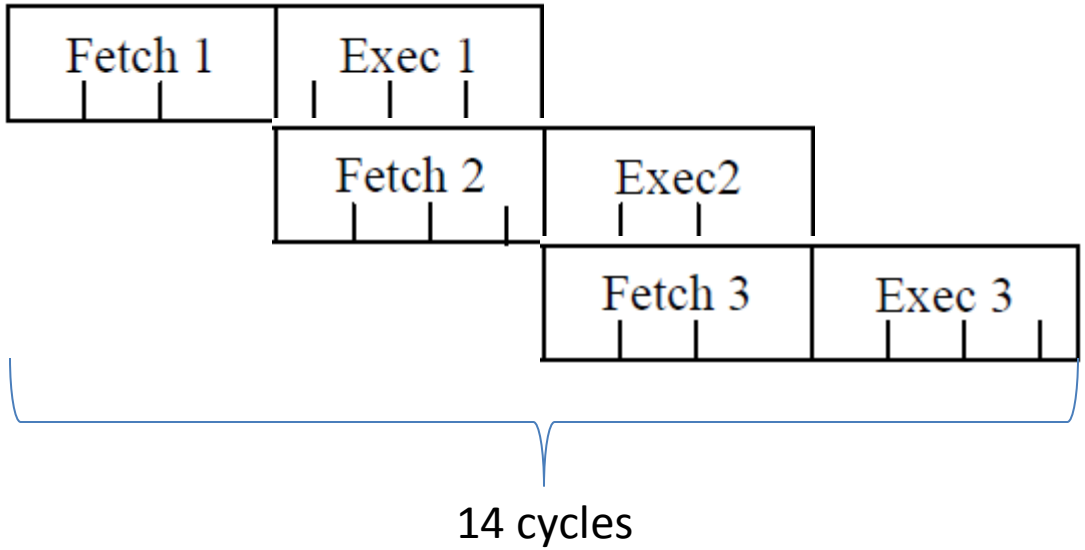




BIU: **Bus Interface Unit**
EU: **Execution Unit**

Gain


Fetch 1
Exec 1
Fetch 2
Exec2
Fetch 3
Exec 3
21 cycles

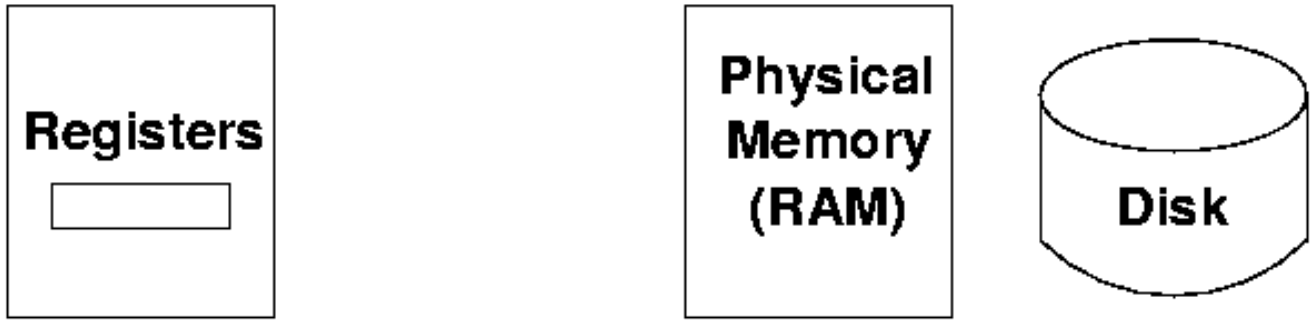

Fetch 1
Exec 1
Fetch 2
Exec2
Fetch 3
Exec 3
14 cycles

Gain percentage

21-14/21 * 100

= 33.334 %

Memory Hierarchy



→ access time decreases

→ amount of storage increases

→ cost per byte decreases

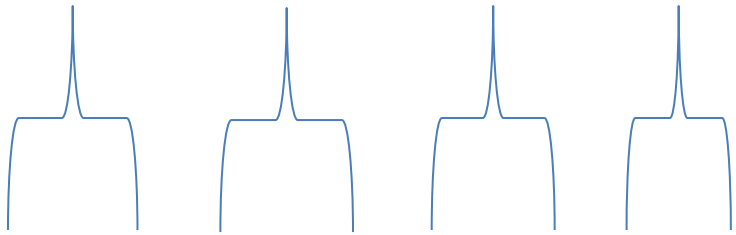
Registers in 8088/8086

Table 1-2: Registers of the 8086/286 by Category

Category	Bits	Register Names
General	16	AX, BX, CX, DX
	8	AH, AL, BH, BL, CH, CL, DH, DL
Pointer	16	SP (stack pointer), BP (base pointer)
Index	16	SI (source index), DI (destination index)
Segment	16	CS (code segment), DS (data segment), SS (stack segment), ES (extra segment)
Instruction	16	IP (instruction pointer)
Flag	16	FR (flag register)

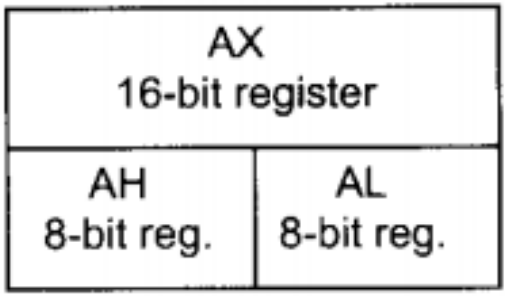
General

AX BX CX DX => 16 bits



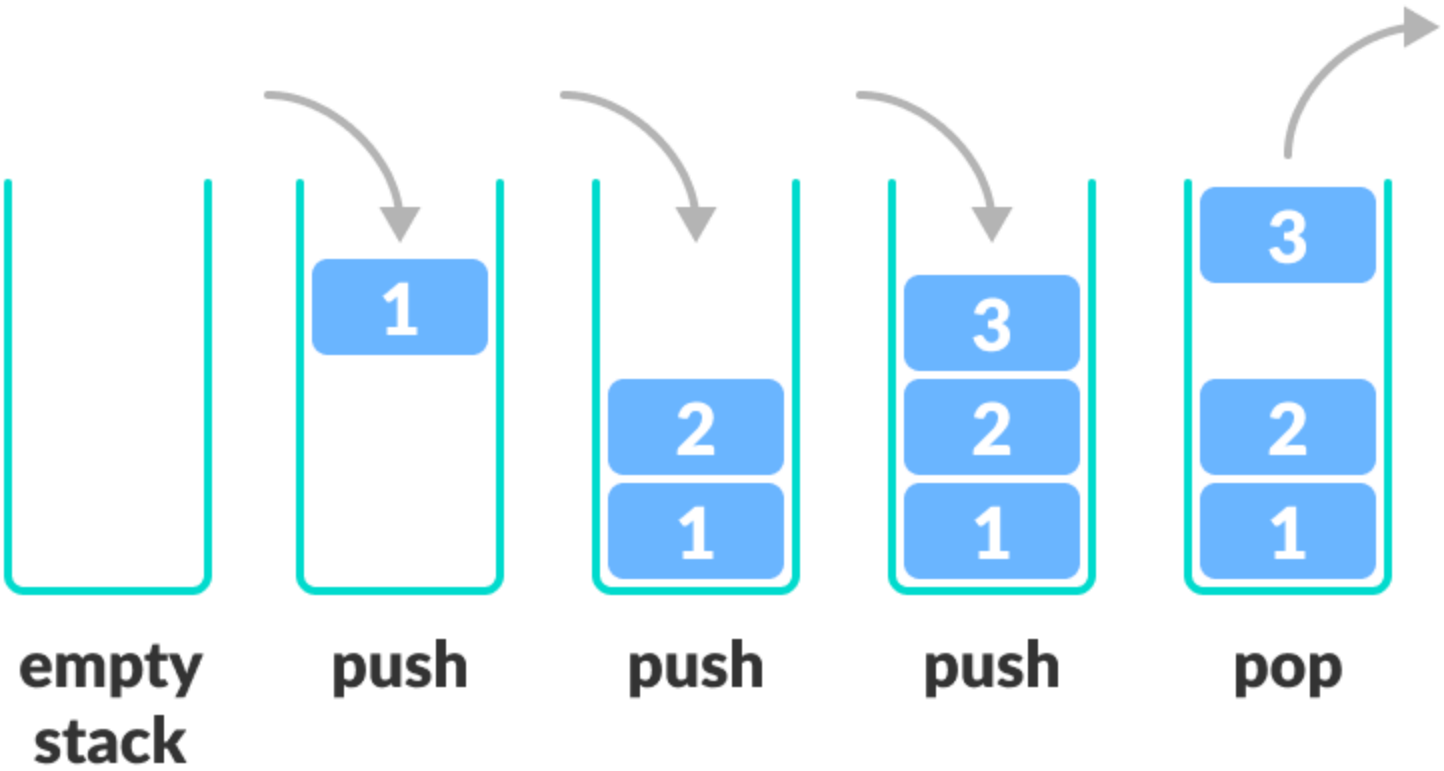
AH AL BH BL CH CL DH DL => 8 bits

For example

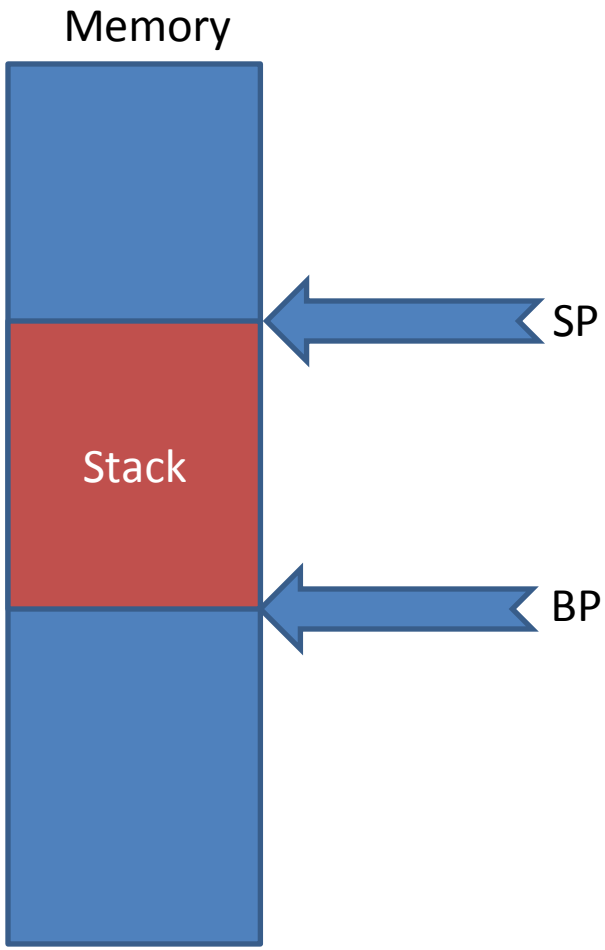


Pointer

SP (Stack Pointer) , BP (Base Pointer) => 16 bits

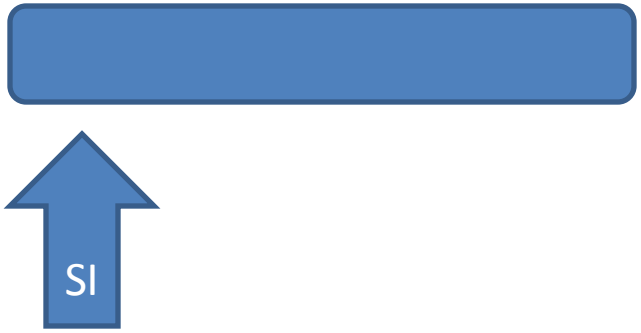

1
2
1
3
2
1
3
2
1
empty stack
push
push
push
pop

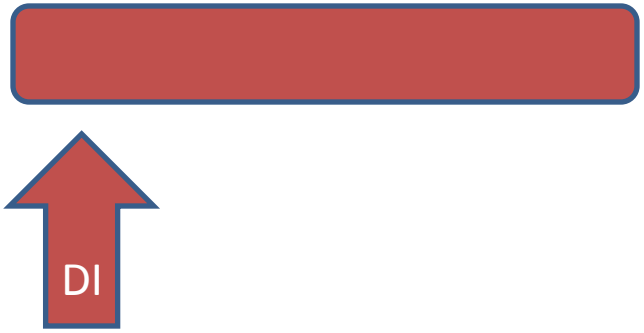
SP (Stack Pointer) , BP (Base Pointer) => 16 bits



Index

SI (Source Index), DI (Destination Index) => 16 bits





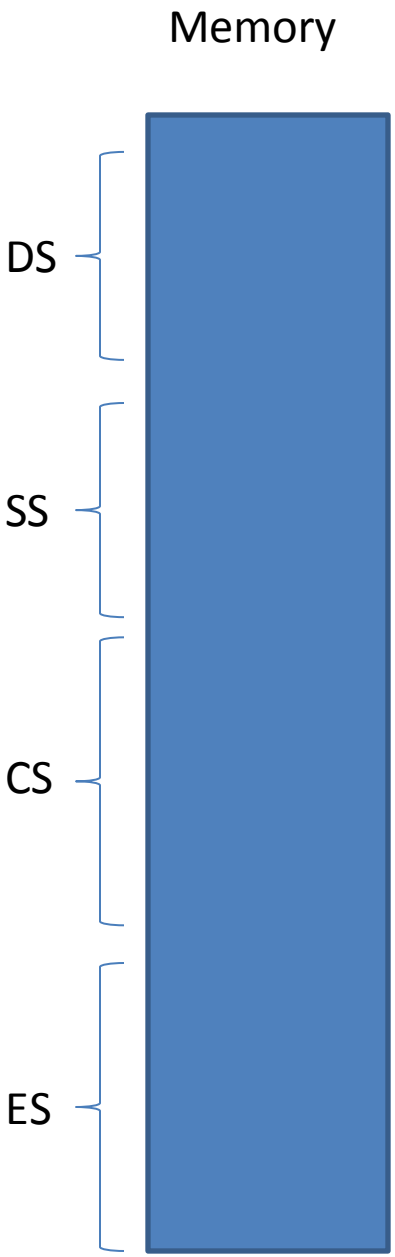
Segment

DS (Data Segment)

SS (Stack Segment)

CS (Code Segment)

ES (Extra Segment)



Instruction Pointer

IP (Instruction Pointer): pointer to the next instruction to be fetched.

Flag

The 16 bits of the flag register:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	R	R	R	OF	DF	IF	TF	SF	ZF	U	AF	U	PF	U	CF

R =	reserved	SF =	sign flag
U =	undefined	ZF =	zero flag
OF =	overflow flag	AF =	auxiliary carry flag
DF =	direction flag	PF =	parity flag
IF =	interrupt flag	CF =	carry flag
TF =	trap flag		

Figure 1-5. Flag Register

2. Name the general-purpose registers of the 8088/86.

(a) 8-bit (b) 16-bit

3. Which of the following registers cannot be split into high and low bytes?

(a) CS (b) AX (c) DS (d) SS (e) BX (f) DX (g) CX (h) SI (i) DI

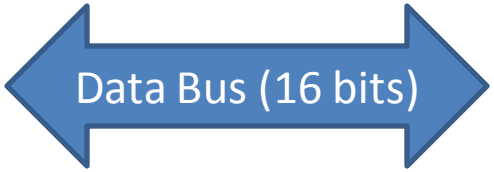
2. a.AH,Al,BH,BL,CH,CL,DH,DL

b. AX,BX,CX,DX

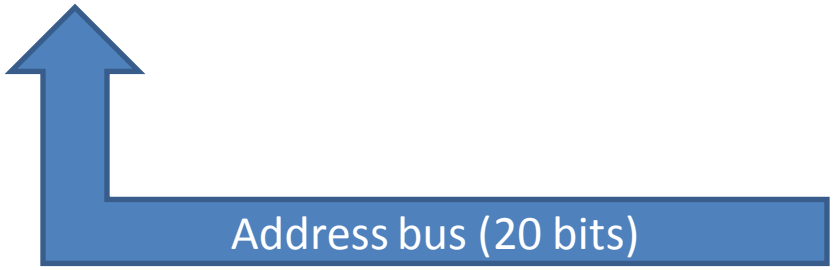
3.

Memory of 8086/8088

Address (20 bits)	data
0	Byte
1	Byte
2	Byte
3	Byte
4	Byte
5	Byte
6	Byte



CPU



Address bus: 20 bits (memory=1MB)

Data bus, registers: 16 bits

How does the CPU deal with an address if the address bus is 20 bits and maximum register is 16 bits?

Address bus: 20 bits (memory=1MB)

Data bus, registers: 16 bits

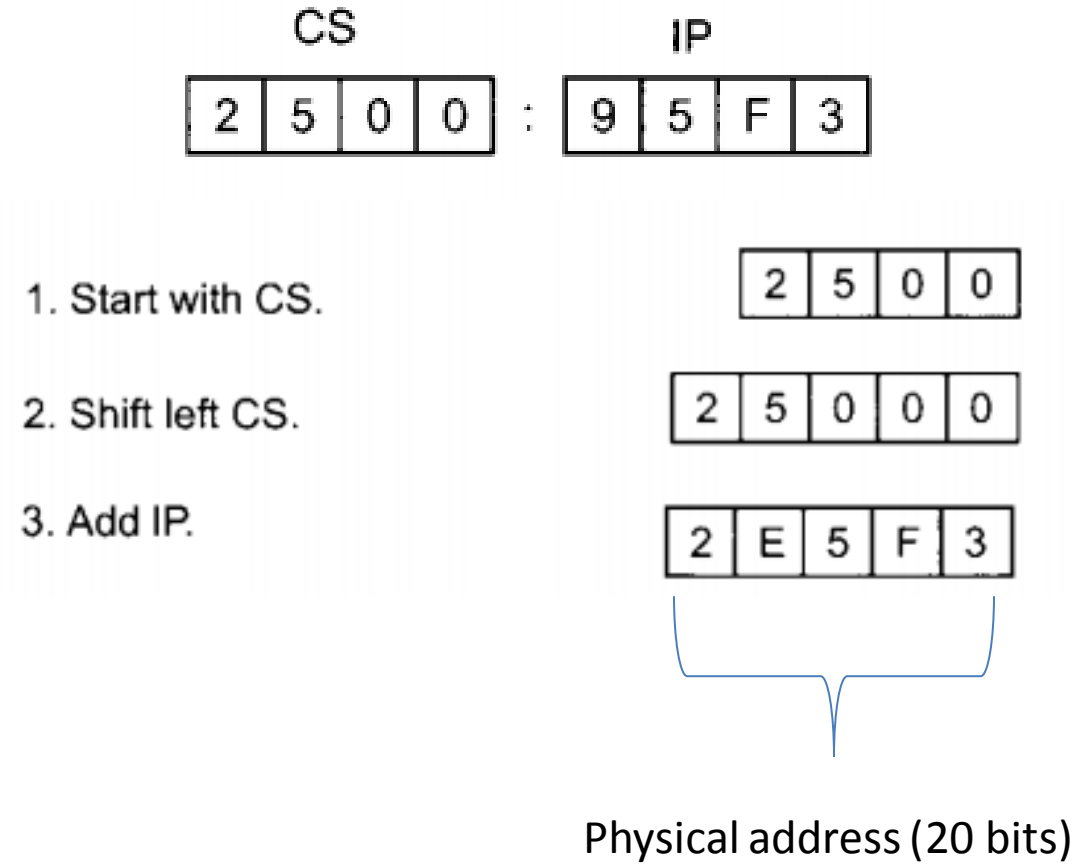
How does the CPU deal with an address if the address bus is 20 bits and maximum register is 16 bits?

It uses 2 registers

Segment Register : Offset Register

Table 1-3: Offset Registers for Various Segments

Segment register:	CS	DS	ES	SS
Offset register(s):	IP	SI, DI, BX	SI, DI, BX	SP, BP


CS
IP
2 5 0 0 : 9 5 F 3
1. Start with CS.
2 5 0 0
2. Shift left CS.
2 5 0 0 0
3. Add IP.
2 E 5 F 3
Physical address (20 bits)

DS=1771H, Offset=202AH

- Logical Address
- Physical Address
- Range of this segment register (DS)

DS=1771H, Offset=202AH

- Logical Address
- Physical Address
- Range of this segment register (DS)

Answer:

Logical address: 1771:202A

Physical address:

$$\begin{array}{r} 17710 \\ +\ \ 202A \\ \hline 1973A \end{array}$$

DS=1771H, Offset=202AF

- Logical Address
- Physical Address
- Range of this segment register (DS)

Answer

Lower Range:	Upper Range:
Logical = 1771:0000 Physical=17710+00000=17710	Logical = 1771:FFFF Physical = 17710+0FFFF=2770F

Example 1-1

If CS = 24F6H and IP = 634AH, show:
(a) The logical address
(b) The offset address
and calculate:
(c) The physical address
(d) The lower range
(e) The upper range of the code segment

Solution:
(a) 24F6:634A
(b) 634A
(c) 2B2AA (24F60 + 634A)
(d) 24F60 (24F60 + 0000)
(e) 34F5F (24F60 + FFFF)

Example 1-2

Assume that DS is 5000 and the offset is 1950. Calculate the physical address of the byte.

Solution:

DS				:	offset			
5	0	0	0	:	1	9	5	0

The physical address will be 50000 + 1950 = 51950.

1. Start with DS.

5	0	0	0

2. Shift DS left.

5	0	0	0	0

3. Add the offset.

5	1	9	5	0

Assume that the DS register is 578C. To access a given byte of data at physical memory location 67F66, does the data segment cover the range where the data is located?

Assume that the DS register is 578C. To access a given byte of data at physical memory location 67F66, does the data segment cover the range where the data is located?

The range

From 578C:0000 = 578C0+00000 = 578C0

To 578C:FFFF = 578C0+0FFFF = 678BF

So not in range.

The MOV instruction

Mov destination, source

Examples

```
MOV   CL,55H      ;move 55H into register CL
MOV   DL,CL       ;copy the contents of CL into DL (now DL=CL=55H)
MOV   AH,DL       ;copy the contents of DL into AH (now AH=DL=55H)
MOV   AL,AH       ;copy the contents of AH into AL (now AL=AH=55H)
MOV   BH,CL       ;copy the contents of CL into BH (now BH=CL=55H)
MOV   CH,BH       ;copy the contents of BH into CH (now CH=BH=55H)
```

The use of 16-bit registers is demonstrated below.

```
MOV   CX,468FH    ;move 468FH into CX (now CH=46,CL=8F)
MOV   AX,CX       ;copy contents of CX to AX (now AX=CX=468FH)
MOV   DX,AX       ;copy contents of AX to DX (now DX=AX=468FH)
MOV   BX,DX       ;copy contents of DX to BX (now BX=DX=468FH)
MOV   DI,BX       ;now DI=BX=468FH
MOV   SI,DI       ;now SI=DI=468FH
MOV   DS,SI       ;now DS=SI=468FH
MOV   BP,DI       ;now BP=DI=468FH
```

Mov rules

- The mov must between registers of equal sizes: mox Ax, DL is wrong
- Values cannot be moved directly to segment registers (CS, DS, SS, ES): mov DS, 78 is wrong

  Mov Ax,78

  Mov Ds,Ax
- Moving values larger than registers will cause an error: mov AL, 79ABH is wrong

Which of the following instructions are wrong ?

(a) MOV AX, 27	(b) MOV AL, 97F	(c) MOV DS, 9BF2
(d) MOV CX, 397	(e) MOV SI, 9516	(f) MOV CS, 3490
(g) MOV DS, BX	(h) MOV BX, CS	(i) MOV CH, AX
(j) MOV AX, 23FB9	(k) MOV CS, BH	(l) MOV AX, DL

Wrong: b, c, f, i, j, k, L